

Translate text and speech

Prerequisites

Create a Microsoft Foundry project

Explore Azure Translator in Foundry Tools in the portal

Get application files from GitHub

Create a text translation application

Create a speech translation application

Clean up resources

Azure Translator in Foundry Tools is a service that enables you to translate text between languages. Similarly, **Azure Speech in Foundry Tools** provides translation services for speech. In this exercise, you'll use them to create translation apps that translates input in any supported language to the target language of your choice.

While this exercise is based on Python, you can develop text translation applications using multiple language-specific SDKs; including:

- [Azure Translator client library for Python](#)
- [Azure Translator client library for .NET](#)
- [Azure Translator client library for JavaScript](#)
- [Azure AI Speech SDK for Python](#)
- [Azure AI Speech SDK for .NET](#)
- [Azure AI Speech SDK for JavaScript](#)

This exercise takes approximately **30** minutes.

Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](#)
- [Visual Studio Code](#) installed
- [Python version 3.13.xx](#) installed*
- [Git](#) installed and configured
- [Azure CLI](#) installed

* Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.
2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - **Foundry resource:** Use the default name for your resource (usually {project_name}-resource)*
 - **Subscription:** Your Azure subscription
 - **Resource group:** Create or select a resource group
 - **Region:** Select any available region

Note: Use a recommended Microsoft Foundry region. Model availability may vary by region.

! **TIP:** * Remember the Foundry resource name - you'll need it later!

3. Wait for your project to be created. Then view the home page for your project.

Explore Azure Translator in Foundry Tools in the portal

You can use the Azure Translator playground in the Foundry portal to experiment with the service.

1. Now you're ready to **Start building**. Select **Explore playgrounds** (or on the **Build** page, select the **Models** tab) to view the models in your project.
2. In the **Models** page, select the **AI services** tab to view the list of Azure services in Foundry Tools.
3. In the list of tools, select **Azure Translator - Text translation**.
4. In the Text translator playground, in the **Source text** area, enter the text `Hello world!`. Then, in the **Translation** area, select any language and use the **Translate** button to generate the translation.
5. Try a few more languages.
6. Select the **Code** tab to view sample code for using Azure Translator; and note the **ENDPOINT** variable used in the code for the REST API, which should be similar to `https://{foundry-resource-name}.cognitiveservices.azure.com/`.

This endpoint uses an older format for Azure AI Services, but is still used to connect to the Azure Translator resource in a Foundry resource. You can also use it to connect to Azure Speech tools.

! **TIP:** You're going to need the endpoint later!

Get application files from GitHub

The initial application files you'll need to develop the translation application are provided in a GitHub repo.

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

3. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
4. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

! **Tip:** If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */Labfiles/07-translation/Python/translators* folder; but it's OK if you don't - we'll install them later!

! **Tip:** If you prefer to use the terminal, you can create your **Venv** environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

Create a text translation application

Now you're ready to use Azure Translator to implement text translation.

1. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/07-translation/Python/translators**. The application files include:
 - **.env** (the application configuration file)
 - **requirements.txt** (the Python package dependencies that need to be installed)
 - **translate-text.py** (the code file for text-application)
 - **translate-speech.py** (the code file for speech-application)

Configure your text translation application

1. In the **Explorer** pane, in the **translators** folder, select the **.env** file to open it. Then update the configuration values to reflect the Cognitive Services **endpoint** for your Foundry resource.

Important: The endpoint should be *https://{YOUR_FOUNDRY_RESOURCE}.cognitiveservices.azure.com/*. The Foundry Resource name usually takes the form *{project_name}-resource*.

Save the modified configuration file.

2. In the **Explorer** pane, right-click the **translators** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */Labfiles/07-translation/Python/translators* folder.)

Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

3. Ensure that the terminal is open in the **translators** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
4. Install the Azure Translator SDK, Speech SDK, and other required packages by running the following command:

CodeCopy

```
pip install -r requirements.txt
```

Add code to translate text

1. In the **Explorer** pane, in the **translators** folder, open the **translate-text.py** file.
2. Review the existing code. You will add code to work with Azure Translator.

Tip: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need to use the Translator SDK:

CodeCopy

```
# import namespaces
from azure.identity import DefaultAzureCredential
from azure.ai.translation.text import *
from azure.ai.translation.text.models import InputTextItem
```

4. In the **main** function, note that the existing code reads the configuration settings.
5. Find the comment **Create client using endpoint and credential** and add the following code:

Code

 Copy

```
# Create client using endpoint and credential
credential = DefaultAzureCredential()
client = TextTranslationClient(credential=credential, endpoint=foundry_endpoint)
```

6. Find the comment **Choose target language** and add the following code, which uses the Text Translator service to return list of supported languages for translation, and prompts the user to select a language code for the target language:

Code

 Copy

```
# Choose target language
languagesResponse = client.get_supported_languages(scope="translation")
print("{} languages supported.".format(len(languagesResponse.translation)))
print("(See https://learn.microsoft.com/azure/ai-services/translator/language-support#translation)")
print("Enter a target language code for translation (for example, 'en'):")
targetLanguage = "xx"
supportedLanguage = False
while supportedLanguage == False:
    targetLanguage = input()
    if targetLanguage in languagesResponse.translation.keys():
        supportedLanguage = True
    else:
        print("{} is not a supported language.".format(targetLanguage))
```

7. Find the comment **Translate text** and add the following code, which repeatedly prompts the user for text to be translated, uses the Azure AI Translator service to translate it to the target language (detecting the source language automatically), and displays the results until the user enters *quit*:

Code

 Copy

```
# Translate text
inputText = ""
while inputText.lower() != "quit":
    inputText = input("Enter text to translate ('quit' to exit):")
    if inputText != "quit":
        input_text_elements = [InputTextItem(text=inputText)]
        translationResponse = client.translate(body=input_text_elements, to_language=[targetLanguage])
        translation = translationResponse[0] if translationResponse else None
        if translation:
            sourceLanguage = translation.detected_language
            for translated_text in translation.translations:
                print(f"'{inputText}' was translated from {sourceLanguage.language} to {translated_text.to} as '{translated_text.text}'.")
```

8. Save the changes to the code file. Then, in the terminal pane, use the following command to sign into Azure.

Code

 Copy

```
az login
```

Note: In most scenarios, just using `az login` will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the `--tenant` parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

9. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

10. After you have signed in, enter the following command to run the application:

Code

```
python translate-text.py
```

Copy

11. When prompted, enter a valid target language from the list in the link displayed.

12. Enter a phrase to be translated (for example `This is a test` or `C'est un test`) and view the results, which should detect the source language and translate the text to the target language.

13. When you're done, enter `quit` . You can run the application again and choose a different target language.

Create a speech translation application

Now you're ready to use Azure Speech to implement text translation.

Configure your speech translation application

1. In the **translators** folder, verify that the `.env` file contains the **endpoint** for your Foundry resource (Azure Speech can use the same information as Azure Translator to connect to your Foundry resource).
2. Ensure that the terminal is open in the **translators** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
3. If you did not previously install the required packages, enter the following command to do so now:

Code

```
pip install -r requirements.txt
```

Copy

Add code to translate speech

1. In the **Explorer** pane, in the **translators** folder, open the **translate-speech.py** file.
2. Review the existing code. You will add code to work with Azure Speech.

Tip: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the Speech SDK:

Code

```
# Import namespaces
from azure.identity import DefaultAzureCredential
import azure.cognitiveservices.speech as speech_sdk
```

Copy

4. In the **main** function, under the comment **Get configuration settings**, note that the code loads the endpoint you defined in the configuration file.
5. Find the following code under the comment **Configure translation**, and add the following code to configure your connection to the Foundry endpoint for Azure Speech, and prepare to translate speech in US English to French, Spanish, and Hindi:

CodeCopy

```
# Configure translation
credential = DefaultAzureCredential()
translation_cfg = speech_sdk.translation.SpeechTranslationConfig(
    token_credential=credential,
    endpoint=foundry_endpoint
)
translation_cfg.speech_recognition_language = 'en-US'
translation_cfg.add_target_language('fr')
translation_cfg.add_target_language('es')
translation_cfg.add_target_language('hi')
audio_in_cfg = speech_sdk.AudioConfig(use_default_microphone=True)
translator = speech_sdk.translation.TranslationRecognizer(
    translation_config=translation_cfg,
    audio_config=audio_in_cfg
)
print('Ready to translate from',translation_cfg.speech_recognition_language)
```

6. You will use the **SpeechTranslationConfig** to translate speech into text, but you will also use a **SpeechConfig** to synthesize translations into speech. Add the following code under the comment **Configure speech for synthesis of translations**:

CodeCopy

```
# Configure speech for synthesis of translations
speech_cfg = speech_sdk.SpeechConfig(
    token_credential=credential, endpoint=foundry_endpoint)
voices = {
    "fr": "fr-FR-HenriNeural",
    "es": "es-ES-ElviraNeural",
    "hi": "hi-IN-MadhurNeural"
}
print('Ready to use speech service.')
```

7. Now it's time to add the code to translate the user's speech into the system microphone. Find the comment **Translate user speech**, and add the following code:

CodeCopy

```
# Translate user speech
print("Speak now...")
translation_results = translator.recognize_once_async().get()
print(f"Translating '{translation_results.text}'")
```

8. When the results are returned, the application will iterate through the translations, printing the text and playing the synthesized speech through the default system speaker. Find the comment **Print and speak the translation results** and add the following code:

CodeCopy

```
# Print and speak the translation results
translations = translation_results.translations
for translation_language in translations:

    print(f"{translation_language}: '{translations[translation_language]}'")

    speech_cfg.speech_synthesis_voice_name = voices.get(translation_language)
    audio_out_cfg = speech_sdk.audio.AudioOutputConfig(use_default_speaker=True)
    speech_synthesizer = speech_sdk.SpeechSynthesizer(speech_cfg, audio_out_cfg)
    speak = speech_synthesizer.speak_text_async(translations[translation_language]).get()

    if speak.reason != speech_sdk.ResultReason.SynthesizingAudioCompleted:
        print(speak.reason)
```

9. Save the changes to the code file. Then, in the terminal pane, if you are not already signed into Azure (or your session may have expired) use the following command to sign into Azure.

Code Copy

```
az login
```

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

10. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

11. After you have signed in, enter the following command to run the application:

Code Copy

```
python translate-speech.py
```

12. When prompted, say something aloud (for example, "Hello!").

The program should translate it to the languages specified in the code (French, Spanish, and Hindi), and print and speak the translations.

NOTE: The translation to Hindi may not always be displayed correctly in the terminal due to character encoding issues.

Clean up resources

If you have finished exploring Microsoft Foundry, delete any resources that you no longer need. This avoids accruing any unnecessary costs.

1. Open the **Azure portal** at <https://portal.azure.com> and select the resource group that contains the resources you created.
2. Select **Delete resource group** and then **enter the resource group name** to confirm. The resource group is then deleted.

